

Software Engineering Unit 1 - Notes

1. Introduction to Software Engineering (SE) & Motivation

1.1 Defining Software Engineering

According to Ian Sommerville, Software Engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use. Roger S. Pressman defines it as a layered technology consisting of a "quality focus," "processes," "methods," and "tools." It is the establishment and use of sound engineering principles to obtain economically software that is reliable and works efficiently on real machines.

1.2 The Software Crisis and Motivation for SE

The term "Software Crisis" was coined in the late 1960s to describe the rapid increase in computing power and the complexity of the problems that could be tackled, which outpaced the ability of programmers to write reliable software. The motivation for formalized SE stems from this crisis.

- **Managing Escalating Complexity:** As outlined by Pressman, software is logical rather than physical. It does not "wear out" like hardware, but it deteriorates due to continuous changes. Engineering practices provide structural models to manage this logical complexity.
- **Cost and Schedule Overruns:** Unstructured programming often leads to unpredictable delivery times and budget explosions. SE introduces quantifiable metrics and project management techniques to align software development with business constraints.
- **Quality and Reliability:** Sommerville emphasizes that we now depend on software for critical infrastructure (aviation, medical, finance). Software engineering introduces rigorous Verification and Validation (V&V) to ensure dependability.
- **Maintainability:** A majority of software costs are incurred after the software has been deployed. SE focuses on creating readable, modular, and well-documented code to facilitate long-term maintenance.

2. Case Study: The CrowdStrike Outage & SE Implications

2.1 Event Overview

The CrowdStrike incident serves as a modern, critical case study of software engineering failures in high-stakes environments. A flawed configuration update to the Falcon sensor

software caused millions of Windows systems worldwide to enter a "Blue Screen of Death" (BSOD) boot loop, halting global airlines, hospitals, and financial institutions.

2.2 Analysis Through the Lens of Pressman and Sommerville

- **Failure in V&V (Sommerville):** Sommerville's text stresses that critical systems must undergo strict static and dynamic validation. The CrowdStrike update bypassed comprehensive dynamic testing that would have caught the null-pointer exception or memory access violation prior to deployment.
- **Deployment and Configuration Management (Pressman):** Pressman highlights Software Configuration Management (SCM) as an umbrella activity applied throughout the software process. The failure to treat configuration updates with the same rigor as compiled code updates represents a fundamental breakdown in SCM.
- **Lack of Phased Rollouts (Canary Testing):** Modern SE dictates that updates with system-level privileges should be deployed in concentric rings (Canary deployments). Pushing a kernel-level update globally at once violates core reliability engineering principles.
- **Resiliency and Rollback:** The software lacked a safe-mode fallback. According to SE principles of dependable systems, a critical component must fail safely (fail-secure or fail-safe). The system's inability to automatically revert the corrupt configuration file meant manual intervention was required on every single machine.

3. Software Development Life Cycles (SDLCs)

3.1 The Waterfall Model

Introduced by Winston Royce, the Waterfall model is the classic life cycle. Pressman describes it as a systematic, sequential approach that begins with customer specification and progresses through planning, modeling, construction, and deployment.

- **Pros:** Highly structured, easy to manage, clear milestones, works well when requirements are well understood and fixed (e.g., embedded systems).
- **Cons:** Real projects rarely follow the sequential flow. Sommerville notes that changing requirements cannot be easily accommodated. A working version of the program is not available until late in the project timeline.

3.2 The V-Model

The V-Model is a variation of Waterfall that demonstrates the relationship between each phase of the development life cycle and its associated phase of testing. As the team moves down the left side (requirements, architecture, design), they define the tests that will be executed on the right side (integration, system, acceptance testing).

- **Pros:** Proactive defect tracking, emphasizes V&V at early stages, ideal for mission-critical systems.
- **Cons:** Highly rigid, heavily reliant on upfront documentation, poor adaptability to dynamic

business needs.

3.3 The Spiral Model

Developed by Barry Boehm, the Spiral Model couples the iterative nature of prototyping with the controlled and systematic aspects of the Waterfall model. It provides the potential for rapid development of increasingly more complete versions of the software.

- **Key Concept - Risk Analysis:** According to Sommerville, each loop in the spiral represents a phase of the software process. The distinguishing feature is the explicit consideration of risk. Before starting a phase, risks are assessed, and prototyping is used to mitigate them.
- **Pros:** Excellent for high-risk, large-scale systems; continuous customer evaluation.
- **Cons:** Requires considerable risk assessment expertise; complex to manage; not suitable for small projects.

4. Security Development Life Cycle (SecDLC)

4.1 The Need for SecDLC

Traditional SDLCs treat security as a non-functional requirement tested at the end of the project. The SecDLC integrates security checkpoints, or "touchpoints," into every phase of the development process. This embodies the "Shift-Left" philosophy—moving security earlier in the timeline to reduce the cost and impact of vulnerabilities.

4.2 SecDLC Phases and Touchpoints

SDLC Phase	Security Activity (Touchpoint)	Details
Requirements	Security Requirements Elicitation	Defining abuse cases, compliance standards (GDPR, HIPAA), and cryptographic requirements.
Design	Threat Modeling & Risk Analysis	Analyzing the architecture using frameworks like STRIDE to identify potential attack vectors before coding begins.
Implementation	Secure Coding & SAST	Applying secure coding guidelines (e.g., OWASP Top 10) and using Static Application Security Testing tools to analyze source code.

SDLC Phase	Security Activity (Touchpoint)	Details
Testing	DAST & Penetration Testing	Dynamic Application Security Testing and ethical hacking to find runtime vulnerabilities and logic flaws.
Deployment/Maintenance	Continuous Monitoring & Patching	Setting up Intrusion Detection Systems (IDS), managing incident response, and applying security patches rapidly.

5. Agile Methodology and Scrum

5.1 Motivation for Agile

Sommerville points out that the operational environment of most businesses is highly dynamic. Plan-driven models (like Waterfall) fail because requirements change before the system is deployed. Agile methods were developed to reduce the overhead in the software process and to be able to respond quickly to changing requirements.

5.2 Core Agile Principles (The Agile Manifesto)

Pressman discusses how Agile stresses human factors and relies heavily on team autonomy. The core values dictate:

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

5.3 The Scrum Framework in Detail

Scrum is a specific agile process framework characterized by time-boxed iterations called Sprints.

- **Roles:**
 - *Product Owner*: Represents the stakeholders, prioritizes the Product Backlog, and ensures the team delivers value.
 - *Scrum Master*: The servant-leader who enforces Scrum rules, facilitates meetings, and removes impediments.
 - *Development Team*: A cross-functional, self-organizing group of professionals who build the product increment.
- **Artifacts:**

- *Product Backlog*: A dynamic, prioritized list of everything that might be needed in the product (managed by the PO).
- *Sprint Backlog*: The set of Product Backlog items selected for the Sprint, plus a plan for delivering the Increment.
- *Increment*: The sum of all completed backlog items during a sprint that meets the "Definition of Done."
- **Ceremonies (Events)**:
 - *Sprint Planning*: Defining what can be delivered and how the work will be achieved.
 - *Daily Scrum (Stand-up)*: A 15-minute daily synchronization meeting.
 - *Sprint Review*: A demonstration of the working increment to stakeholders to gather feedback.
 - *Sprint Retrospective*: A meeting for the team to inspect itself and plan improvements for the next sprint.

6. Requirements Engineering (RE)

6.1 Introduction to RE

Sommerville defines Requirements Engineering as the process of understanding and defining what services are required from the system and identifying the constraints on the system's operation and development. Pressman highlights that RE provides the appropriate mechanism for understanding what the customer wants, analyzing needs, assessing feasibility, negotiating a reasonable solution, specifying the solution unambiguously, and validating the specification.

6.2 The RE Process

- **Elicitation (Requirements Discovery)**: Interacting with stakeholders to discover their needs. Pressman suggests techniques like collaborative requirements gathering, Quality Function Deployment (QFD), and user scenarios. Sommerville adds ethnographic studies (observing how people actually work rather than how they say they work).
- **Analysis and Negotiation**: Checking requirements for necessity, consistency, and completeness. Conflicts between stakeholders are resolved through negotiation.
- **Specification**: Translating the gathered information into a formal document (SRS).
- **Validation**: Examining the document to ensure it reflects the true needs of the user (reviewing, prototyping, model validation).

6.3 Types of Requirements

- **Functional Requirements**: Statements of services the system should provide, how the system should react to particular inputs, and how it should behave in particular situations (e.g., "The system shall allow users to search for books by author").
- **Non-Functional Requirements**: Constraints on the services or functions offered by the system, such as timing constraints, constraints on the development process, and standards. They often apply to the system as a whole (e.g., "The system shall process a

search query in under 2 seconds").

- **Security Requirements:** A subset of NFRs dealing with Confidentiality, Integrity, and Availability (CIA triad).

7. SRS, RTM, and Requirement Change Management

7.1 Software Requirement Specification (SRS)

The SRS is the official statement of what the system developers should implement. According to IEEE standards referenced by Pressman, a high-quality SRS must be:

- **Unambiguous:** Every requirement has only one interpretation.
- **Complete:** All significant requirements of concern to the user are included.
- **Consistent:** No requirements conflict with one another.
- **Verifiable:** There exists some cost-effective process that can check whether the final software meets the requirement.
- **Traceable:** The origin of each requirement is clear and can be mapped to project assets.

7.2 Requirement Traceability Matrix (RTM)

An RTM is a table that links requirements to their origins and traces them throughout the project life cycle. It proves that every requirement defined in the SRS has been implemented and tested. It maps Requirements IDs to Design Document IDs, Code Module IDs, and Test Case IDs, preventing missing features and "gold-plating" (adding undocumented features).

7.3 Requirement Change Management

Sommerville dictates that requirements will inevitably change as the environment changes or as stakeholders understand their needs better. Change management includes:

1. **Problem Analysis and Change Specification:** The process begins with an identified problem or a proposed change.
2. **Change Analysis and Costing:** The effect of the change on the SRS, design, schedule, and budget is assessed.
3. **Change Implementation:** If approved by the Change Control Board (CCB), the SRS and all subsequent artifacts are formally updated.

8. UML Use Case Diagrams, Personas, and Scenarios

8.1 Personas and User Scenarios

Pressman advocates for scenario-based modeling. Before drawing diagrams, we must understand the user.

- **Personas:** Detailed fictional representations of target audience segments. They include demographics, technical expertise, goals, and frustrations. Personas ground requirements engineering in empathy and reality.

- **User Scenarios:** Narrative descriptions of a persona interacting with the system to achieve a specific goal. They help identify necessary use cases.

8.2 UML Use Case Diagrams

Introduced by Ivar Jacobson, Use Case modeling is a fundamental part of the Unified Modeling Language (UML). It provides a high-level visual representation of system functionality from an external perspective.

- **Actors:** Represented as stick figures. They are external entities (humans, other systems, time triggers) that interact with the system.
- **Use Cases:** Represented as ovals. They describe a discrete, standalone activity the system performs to yield a valuable result for an actor.
- **System Boundary:** A box defining the scope of the system; actors are outside, use cases are inside.
- **Relationships:**
 - **Association:** A solid line connecting an actor to a use case.
 - **<<include>>:** A dashed arrow indicating that the base use case explicitly and mandatorily incorporates the behavior of another use case (e.g., "Checkout" <<include>> "Calculate Tax").
 - **<<extend>>:** A dashed arrow indicating that a base use case can optionally be extended by another use case under specific conditions (e.g., "Login" <<extend>> "Show Invalid Password Error").

9. Testing Strategies & Security Validation Plan

9.1 Software Testing Fundamentals

Pressman breaks testing down into Verification ("Are we building the product right?") and Validation ("Are we building the right product?"). Testing should proceed from the smallest components outward.

- **Unit Testing:** Focuses verification on the smallest unit of software design (the module or component). Usually executed via White-box testing by developers using tools like JUnit.
- **Integration Testing:** A systematic technique for constructing the software architecture while conducting tests to uncover errors associated with interfacing. Combats the "it works on my machine" problem.
- **System Testing:** Fully integrated software is tested against the entire system requirements (Hardware, OS, databases). Includes performance, stress, and security testing.
- **User Acceptance Testing (UAT):** Validation conducted by the end-user. Sommerville distinguishes between Alpha testing (users test software at the developer's site) and Beta testing (users test software in their own live environment).

9.2 Security Validation Plan

A standard test plan is insufficient for modern adversarial environments. A Security Validation Plan is a formal document outlining how the system will be verified against security requirements.

- **Automated Vulnerability Scanning:** Utilizing automated tools (DAST) to scan compiled applications or running web services for known vulnerabilities like SQL Injection or Cross-Site Scripting (XSS).
- **Penetration Testing (Ethical Hacking):** Engaging security experts to simulate real-world cyberattacks on the system to discover complex logic flaws that automated tools miss.
- **Fuzz Testing:** Providing invalid, unexpected, or random data as inputs to a computer program to monitor for crashes, failing built-in code assertions, or potential memory leaks.
- **Security Review and Audits:** Manual inspection of code (peer reviews) against secure coding standards and verification of compliance with industry regulations.